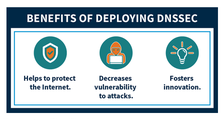
DNSSEC (DNS Security Extensions) – What Is It and Why Is It Important?

This page is available in:
English | العربية (https://www.icann.org/resources/pages/dnssec-what-is-it-why-important-2019-03-20-ar) |
Español (https://www.icann.org/resources/pages/dnssec-what-is-it-why-important-2019-03-20-es) |
Français (https://www.icann.org/resources/pages/dnssec-what-is-it-why-important-2019-03-20-fr) |
Русский (https://www.icann.org/resources/pages/dnssec-what-is-it-why-important-2019-03-20-ru) |
中文 (https://www.icann.org/resources/pages/dnssec-what-is-it-why-important-2019-03-20-zh)

Interested in learning about Domain Name (Domain Name) System Security (Security – Security, Stability and Resiliency (SSR)) Extensions (DNSSEC (DNS Security Extensions))? Click the image below to explore our infographic, which describes what DNSSEC (DNS Security Extensions) is and outlines the benefits of deploying DNSSEC (DNS Security Extensions).



(/en/system/files/files/dnssec-help-secure-info-10jun20-en.pdf)

A brief description of how DNS (Domain Name System) works

To understand Domain Name (Domain Name) System Security (Security – Security, Stability and Resiliency (SSR)) Extensions (DNSSEC (DNS Security Extensions)), it helps to have a basic understanding of the Domain Name (Domain Name) System (DNS (Domain Name System)).

The proper functioning of the Internet is critically dependent on the DNS (Domain Name System) . Every web page visited, every email sent, every picture retrieved from a social media: all those interactions use the DNS (Domain Name System) to translate human-friendly domain names (such as *icann.org*) to the IP (Internet Protocol or Intellectual Property) addresses (such as 192.0.43.7 and 2001:500:88:200::7) needed by servers, routers, and other network devices to route traffic across the Internet to the proper destination.

Using the Internet on any device starts with the DNS (Domain Name System). For example, consider when a user enters a web site name in a browser on their phone. The browser uses the *stub resolver*, which is part of the device's operating system, to begin the process of translating the web site's domain name into an Internet Protocol (Protocol) (IP (Internet Protocol or Intellectual Property)) address. A stub resolver is a very simple DNS (Domain Name System) client that relays an application's request for DNS (Domain Name System) data to a more complicated DNS (Domain Name System) client called a *recursive resolver*. Many network operators run recursive resolvers to handle DNS (Domain Name System) requests, or *queries*, sent by devices on their network. (Smaller operators and organizations sometimes use recursive resolvers on other networks, including recursive resolvers operated as a service for the public, such as Google Public DNS (Domain Name System), OpenDNS, and Quad9.)

The recursive resolver tracks down, or *resolves*, the answer to the DNS (Domain Name System) query sent by the stub resolver. This resolution process requires the recursive resolver to send its own DNS (Domain Name System) queries, usually to multiple different *authoritative name servers*. The DNS (Domain Name System) data for every domain name is stored on an authoritative name server somewhere on the Internet. DNS (Domain Name System) data for a domain is called a *zone*. Some organizations operate their own name servers to publish their zones, but usually organizations outsource this function to third parties. There are different types of organizations that host DNS (Domain Name System) zones on behalf of others, including registrars, registries, web hosting companies, network server providers, just to name a few.

DNS (Domain Name System) by itself is not secure

DNS (Domain Name System) was designed in the 1980s when the Internet was much smaller, and security was not a primary consideration in its design. As a result, when a recursive resolver sends a query to an authoritative name server, the resolver has no way to verify the authenticity of the response. The resolver can only check that a response appears to come from the same IP (Internet Protocol or Intellectual Property) address where the resolver sent the original query. But relying on the source IP (Internet Protocol or Intellectual Property) address of a response is not a strong authentication mechanism, since the source IP (Internet Protocol or Intellectual Property) address of a DNS (Domain Name System) response packet can be easily forged, or *spoofed*. As DNS (Domain Name System) was originally designed, a resolver cannot easily detect a forged response to one of its queries. An attacker can easily masquerade as the authoritative server that a resolver originally queried by spoofing a response that appears to come from that authoritative server. In other words an attacker can redirect a user to a potentially malicious site without the user realizing it.

Recursive resolvers cache the DNS (Domain Name System) data they receive from authoritative name servers to speed up the resolution process. If a stub resolver asks for DNS (Domain Name System) data that the recursive resolver has in its cache, the recursive resolver can answer immediately without the delay introduced by first querying one or more authoritative servers. This reliance on caching has a downside, however: if an attacker sends a forged DNS (Domain Name System) response that is accepted by a recursive resolver, the attacker has *poisoned the cache* of the recursive resolver. The resolver will then proceed to return the fraudulent DNS (Domain Name System) data to other devices that query for it.

As an example of the threat posed by a *cache-poisoning attack*, consider what happens when a user visits their bank's website. The user's device queries its configured recursive name server for the bank web site's IP (Internet Protocol or Intellectual Property) address. But an attacker could have poisoned the resolver with an IP (Internet Protocol or Intellectual Property) address that points not to the legitimate site but to a web site created by the attacker. This fraudulent website impersonates the bank website and looks just the same. The unknowing user would enter their name and password, as usual. Unfortunately, the user has inadvertently provided its banking credentials to the attacker, who could then log in as that user at the legitimate bank web site to transfer funds or take other unauthorized actions.

The DNS (Domain Name System) Security (Security – Security, Stability and Resiliency (SSR)) Extensions (DNSSEC (DNS Security Extensions))

Engineers in the Internet Engineering Task Force (IETF (Internet Engineering Task Force)), the organization responsible for the DNS (Domain Name System) protocol standards, long realized the lack of stronger authentication in DNS (Domain Name System) was a problem. Work on a solution began in the 1990s and the result was the DNSSEC (DNS Security Extensions) Security (Security – Security, Stability and Resiliency (SSR)) Extensions (DNSSEC (DNS Security Extensions)).

DNSSEC (DNS Security Extensions) strengthens authentication in DNS (Domain Name System) using *digital signatures* based on *public key cryptography*. With DNSSEC (DNS Security Extensions), it's not DNS (Domain Name System) queries and responses themselves that are cryptographically signed, but rather DNS (Domain Name System) data itself is signed by the owner of the data.

Every DNS (Domain Name System) zone has a *public/private key pair*. The zone owner uses the zone's *private key* to sign DNS (Domain Name System) data in the zone and generate digital signatures over that data. As the name "private key" implies, this key material is kept secret by the zone owner. The zone's *public key*, however, is published in the zone itself for anyone to retrieve. Any recursive resolver that looks up data in the zone also retrieves the zone's public key, which it uses to *validate* the authenticity of the DNS (Domain Name System) data. The resolver confirms that the digital signature over the DNS (Domain Name System) data it retrieved is valid. If so, the DNS (Domain Name System) data is legitimate and is returned to the user. If the signature does not validate, the resolver assumes an attack, discards the data, and returns an error to the user.

DNSSEC (DNS Security Extensions) adds two important features to the DNS (Domain Name System) protocol:

- *Data origin authentication* allows a resolver to cryptographically verify that the data it received actually came from the zone where it believes the data originated.
- *Data integrity protection* allows the resolver to know that the data hasn't been modified in transit since it was originally signed by the zone owner with the zone's private key.

Trusting DNSSEC (DNS Security Extensions) keys

Every zone publishes its public key, which a recursive resolver retrieves to validate data in the zone. But how can a resolver ensure that a zone's public key itself is authentic? A zone's public key is signed, just like the other data in the zone. However, the public key is not signed by the zone's private key, but by the *parent zone's* private key. For example, the *icann.org* zone's public key is signed by the *org* zone. Just as a DNS (Domain Name System) zone's parent is responsible for publishing a child zone's list of authoritative name servers, a zone's parent is also responsible for vouching for the authenticity of its child zone's public key. Every zone's public key is signed by its parent zone, except for the root zone: it has no parent to sign its key.

The root zone's public key is therefore an important starting point for validating DNS (Domain Name System) data. If a resolver trusts the root zone's public key, it can trust the public keys of top-level zones signed by the root's private key, such as the public key for the *org* zone. And because the resolver can trust the public key for the *org* zone, it can trust public keys that have been signed by their respective private key, such as the public key for *icann.org*. (In actual practice, the parent zone doesn't sign the child zone's key directly--the actual mechanism is more complicated--but the effect is the same as if the parent had signed the child's key.) The sequence of cryptographic keys signing other cryptographic keys is called a *chain of trust*. The public key at the beginning of a chain of trust is a called a *trust anchor*. A resolver has a list of trust anchors, which are public keys for different zones that the resolver trusts implicitly. Most resolvers are configured with just one trust anchor: the public key for the root zone. By trusting this key at the top of the DNS (Domain Name System) hierarchy, a resolve can build a chain of trust to any location in the DNS (Domain Name System) name space, as long as every zone in the path is signed.

Validating and Signing with DNSSEC (DNS Security Extensions)

In order for the Internet to have widespread security, DNSSEC (DNS Security Extensions) needs to be widely deployed. DNSSEC (DNS Security Extensions) is not automatic: right now it needs to be specifically enabled by network operators at their recursive resolvers and also by domain name owners at their zone's authoritative servers. The operators of resolvers and of authoritative servers have different incentives to turn on DNSSEC (DNS Security Extensions) for their systems, but when they do, more users are assured of getting authenticated answers to their DNS (Domain Name System) queries. Quite simply, a user can have assurance that they are going to end up at their desired online destination.

Enabling DNSSEC (DNS Security Extensions) validation in recursive resolvers is easy. In fact, it has been supported by nearly all common resolvers for many years. Turning it on involves changing just a few lines in the resolver's configuration file. From that point forward, when a user asks the resolver for DNS (Domain Name System) information that comes from zones that are signed, and that data has been tampered with, the user will (purposely) get no data back. DNSSEC (DNS Security Extensions) protects the user from getting bad data from a signed zone by detecting the attack and preventing the user from receiving the tampered data.

Signing zones with DNSSEC (DNS Security Extensions) takes a few steps, but there are millions of zones that sign their DNS (Domain Name System) information so that users of validating resolvers can be assured of getting good data. Almost all common authoritative name server software supports signing zones, and many third-party DNS (Domain Name System) hosting providers also support DNSSEC (DNS Security Extensions). Usually, enabling DNSSEC (DNS Security Extensions) for a zone with a hosting provider is quite easy: often it entails little more than clicking a check box.

For a zone owner to deploy DNSSEC (DNS Security Extensions) by signing their zone's data, that zone's parent, and its parent, all the way to the root zone, also need to be signed for DNSSEC (DNS Security Extensions) to be as effective as possible. A continuous chain of signed zones starting at the root zone allows a resolver to build a chain of trust from the root zone to validate data. For example, to effectively deploy DNSSEC (DNS Security Extensions) in the *icann.org* zone, the *org* zone needs to be signed as well as the root zone. Fortunately, the DNS (Domain Name System) root zone has been signed since 2010, and all gTLDs and many ccTLDs are also signed.

There is one more step to complete DNSSEC (DNS Security Extensions) deployment in a zone: the newly signed zone's public key material needs to be sent to the zone's parent. As described earlier, the parent zone signs the child zone's public key, and allows a chain of trust to be built from parent to child.

Today the zone owner usually needs to communicate the zone's public key material to the parent manually. In most cases, that communication happens through the zone owner's registrar. Just as a zone owner interacts with its registrar to make other changes to a zone, such as the list of the zone's authoritative name servers, the zone owner also interacts with the registrar to update the zone's public key material. While this process is currently manual, recently developed protocols are expected to allow this process to be automated in the future.

The next steps for DNSSEC (DNS Security Extensions)

As DNSSEC (DNS Security Extensions) deployment grows, the DNS (Domain Name System) can become a foundation for other protocols that require a way to store data securely. New protocols have been developed that rely on DNSSEC (DNS Security Extensions) and thus only work in zones that are signed. For example, DNS (Domain Name System)-based Authentication of Named Entities (DANE) allows the publication of Transport Layer Security (Security – Security, Stability and Resiliency (SSR)) (TLS) keys in zones for applications such as mail transport. DANE provides a way to verify the authenticity of public keys that does not rely on certificate authorities. New ways of adding privacy to DNS (Domain Name System) queries will be able to use DANE in the future, as well.

In 2018, ICANN (Internet Corporation for Assigned Names and Numbers) changed the trust anchor for the DNS (Domain Name System) root for the first time. Many lessons were learned about DNSSEC (DNS Security Extensions) during that process. Furthermore, many resolver operators became more aware of DNSSEC (DNS Security Extensions) and turned on validation, and the world got to more clearly see how the entire DNSSEC (DNS Security Extensions) system worked. In the coming years, ICANN (Internet Corporation for Assigned Names and Numbers) hopes to see greater adoption of DNSSEC (DNS Security Extensions), both by resolver operators and zone owners. This would mean that more users everywhere could benefit from DNSSEC (DNS Security Extensions)'s strong cryptographic assurance that they are getting authentic DNS (Domain Name System) answers to their queries.